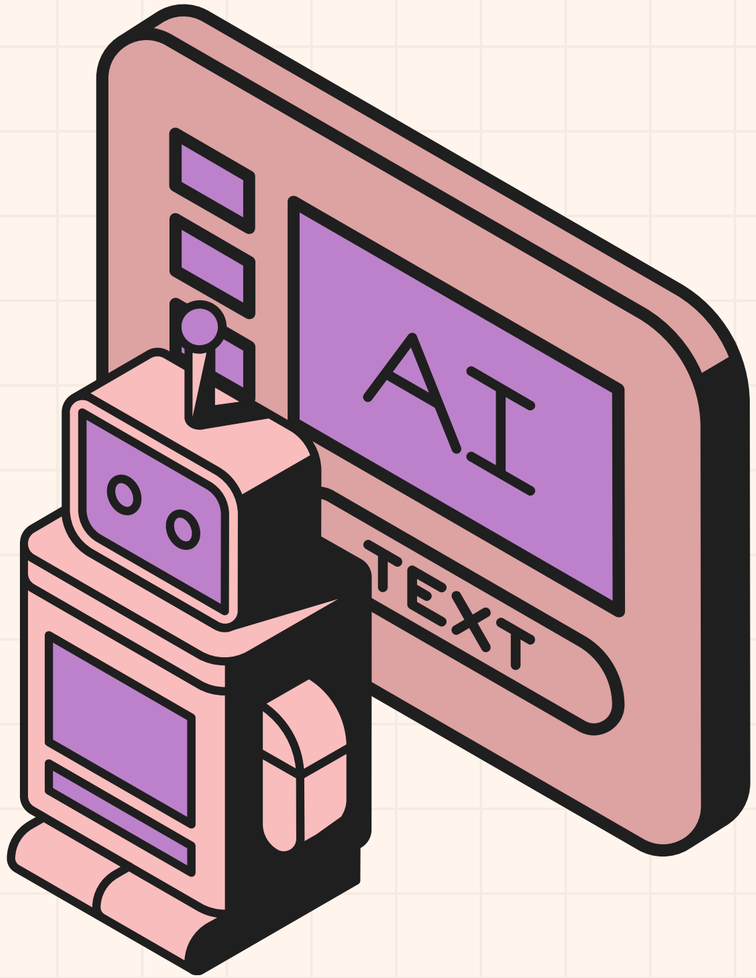
TEAM
OBSIDIAN

OBSIDIANDQ - AN AGENTIC DATA-QUALITY INVESTIGATION AND REMEDIATION SYSTEM



Koushal V
231801084

Jegadeeswaran D
231801069

Karthick S
231801079

Dr. Priyadharsini Ravisankar
Mentor

PROBLEM STATEMENT

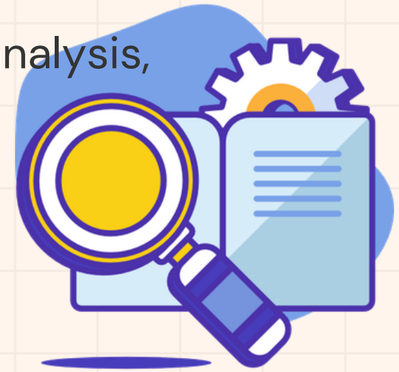
Data quality issues such as missing values, duplicate records, schema drift, and distribution shifts silently degrade the quality of analytics, dashboards, and machine learning models. Although modern data observability tools can detect anomalies, they primarily indicate that a problem exists without explaining its underlying cause. Consequently, data engineers must manually investigate each issue, increasing resolution time and allowing poor-quality data to propagate into downstream systems and business decisions.

KEY CHALLENGES

- Detects, doesn't explain - tools like Great Expectations, Soda, and Monte Carlo flag violations but can't trace them to a root cause or blast radius.
- Manual investigation - engineers must inspect lineage, sample records, and fix transformations by hand for every incident.
- Unsafe automation - giving LLM agents direct DB access risks hallucinated schemas, invalid SQL, and unauthorized writes.
- No real verification - fixes are often assumed successful without re-checking against the original rules.

BUSINESS CASE

- Silent corruption is costly - undetected issues (bad keys, invalid dates, broken joins) quietly degrade dashboards, reports, and ML models.
- Passive tools shift work to humans - Great Expectations, Soda, Monte Carlo alert on failures but leave root-cause tracing and fixes to engineers.
- Naive LLM agents are risky - direct DB write access risks hallucinated schemas, invalid SQL, and destructive changes.
- No real verification or governance - fixes are assumed successful, with no checkpointed human approval before high-risk actions.
- ObsidianDQ's value - AI-driven investigation + deterministic execution → faster root-cause analysis, less manual triage, zero data loss, full audit trail.



USERS

DATA ENGINEERS / ANALYSTS

need fast root-cause explanations, not just alerts

SMALL TEAMS & STARTUPS

can't afford $50K–$200K/year enterprise tools

STUDENTS / RESEARCHERS

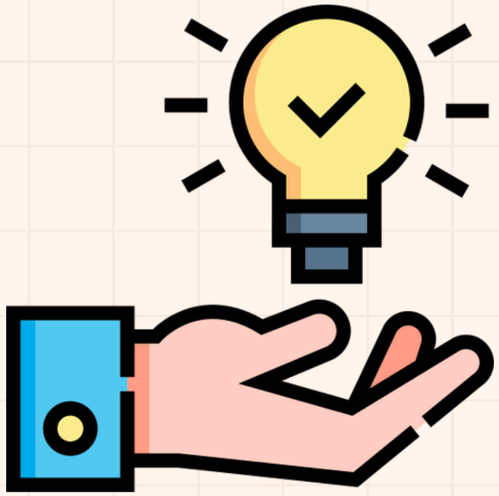
want an explainable, scoped agentic system to learn from

PROPOSED AGENTIC AI SOLUTION

ObsidianDQ takes a hybrid approach that separates LLM reasoning from deterministic execution — the LLM has "eyes, not hands." LLM agents investigate, explain, plan, and triage incidents using read-only tools only, while deterministic components hold exclusive write authority over detection, lineage traversal, SQL transformation, quarantine, and verification. This gives the system the flexibility of AI-driven investigation without exposing production data to unchecked AI writes.

Key capabilities:

- Lineage-based root-cause analysis (DAG traversal, blast-radius calculation)
- AST-grounded SQL healing via sqlglot — not free-text LLM SQL
- Human-in-the-Loop governance for high-risk or low-confidence cases
- Non-destructive remediation (quarantine, not deletion)
- Truthful post-remediation verification — re-runs checks, reports failure honestly



DATASET AND WHY A GENERATED DATASET

ObsidianDQ uses a synthetic e-commerce pipeline as its default dataset, with known faults injected on purpose so results can be graded against a ground truth.

Default dataset

File	Role
stg_orders.parquet	Staging orders table
raw_customers.csv	Upstream customer source
fct_sales.sql	Downstream transformation query
lineage.json	Pipeline dependency graph

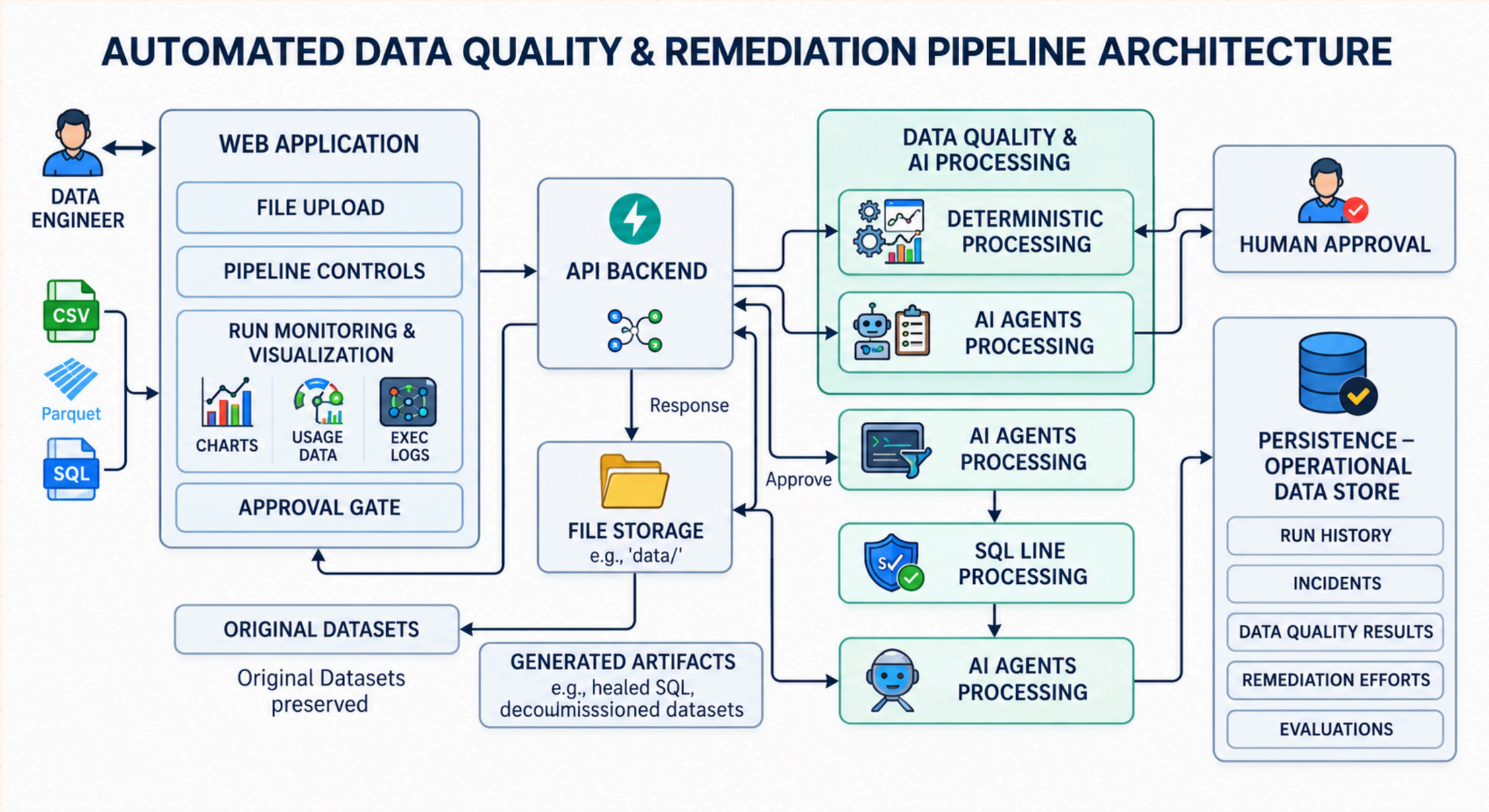
Why synthetic/generated rather than real production data:

- Known ground truth - enables a 20-case benchmark (C01–C20) with individual anomalies, compound anomalies, clean-data controls, and lineage-propagation scenarios.
- Reproducible grading - real production data has no labeled "correct answer" for whether an incident was handled correctly; synthetic data does.
- No sensitive data risk - safe to use in development and live demos.
- Tests honesty, not just success - includes one deliberately unresolvable case (a corrupted historical date) to confirm the system reports VERIFICATION_FAILED truthfully instead of faking success.

TECH STACK

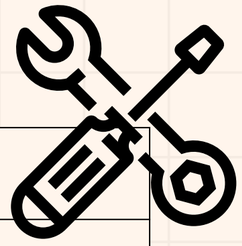
Category	Technology / Details
Backend	FastAPI (REST API layer)
Agent orchestration	LangGraph (stateful multi-agent graph with checkpointing via MemorySaver)
LLM providers	Groq (openai/gpt-oss-20b, primary with tool-calling) → Gemini (gemini-2.5-flash, fallback) → Offline deterministic fallback
SQL parsing & repair	sqlglot (AST-based, DuckDB dialect)
Persistence	DuckDB (embedded analytical database, 6 tables)
Frontend	Next.js 14 (App Router), React, Tailwind CSS, Framer Motion, @xyflow/react (DAG visualization)
Testing	Pytest (49 contract test functions)

ARCHITECTURE DIAGRAM



AGENT WORKFLOW

TOOLS USAGE AND MEMORY USAGE

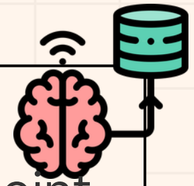


Investigation Agent tools (read-only, via Groq function-calling)

Tool	Purpose
get_sample_rows(column, condition)	Retrieves offending records
get_column_stats(column)	Returns null ratio, cardinality, and data type
get_lineage_context(stage)	Retrieves upstream lineage and downstream blast radius
search_similar_incidents(column, rule)	Searches historical incidents stored in DuckDB

Safety boundary: Investigation Agent only reads — file reads, lineage lookups, DuckDB SELECT. No write/SQL-write tool; all writes happen in a separate node.

Retrieval pattern: structured, tool-based RAG — DuckDB + lineage evidence fed into agent context. No embeddings or vector store; confirmed absent in codebase.



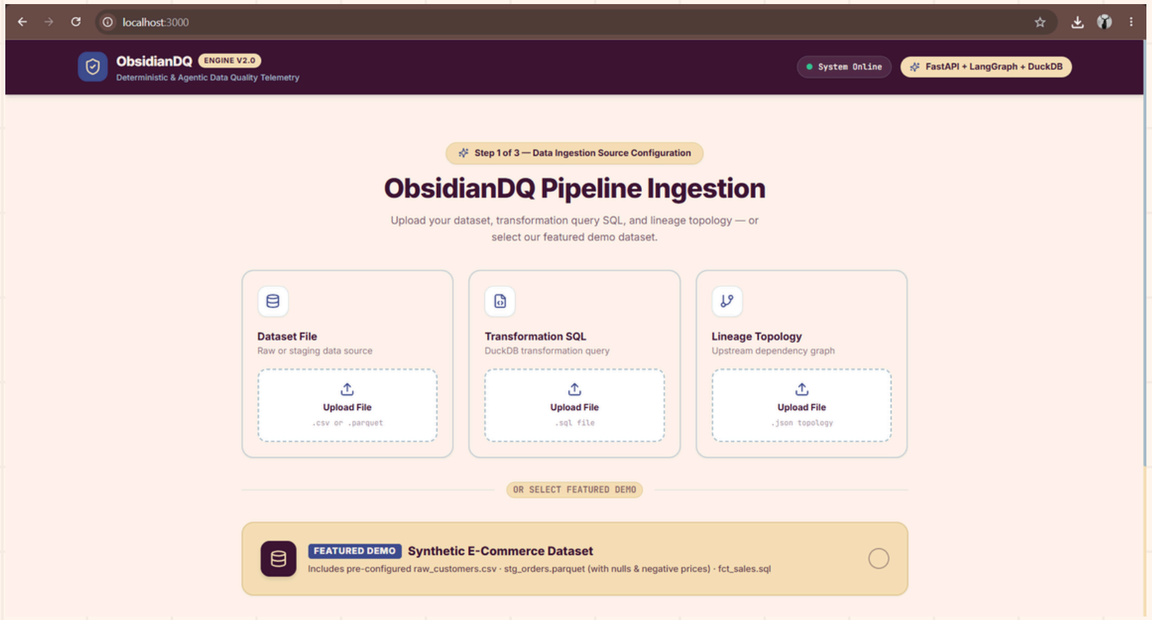
Memory layers

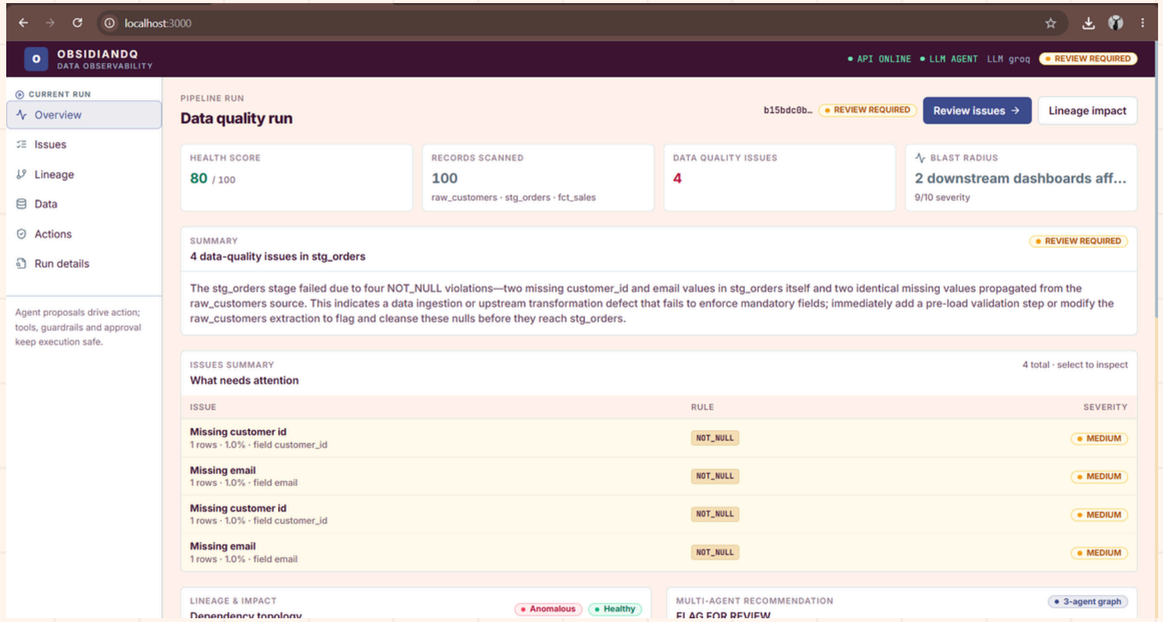
- **Short-term (volatile, in-process) -** RUN_STATES dict + LangGraph MemorySaver checkpoint, both keyed by run_id. Cleared if the backend restarts.
- **Long-term (persistent) -** DuckDB tables: runs, incidents, dq_results, rca_evidence, remediation_results, evaluation_records.

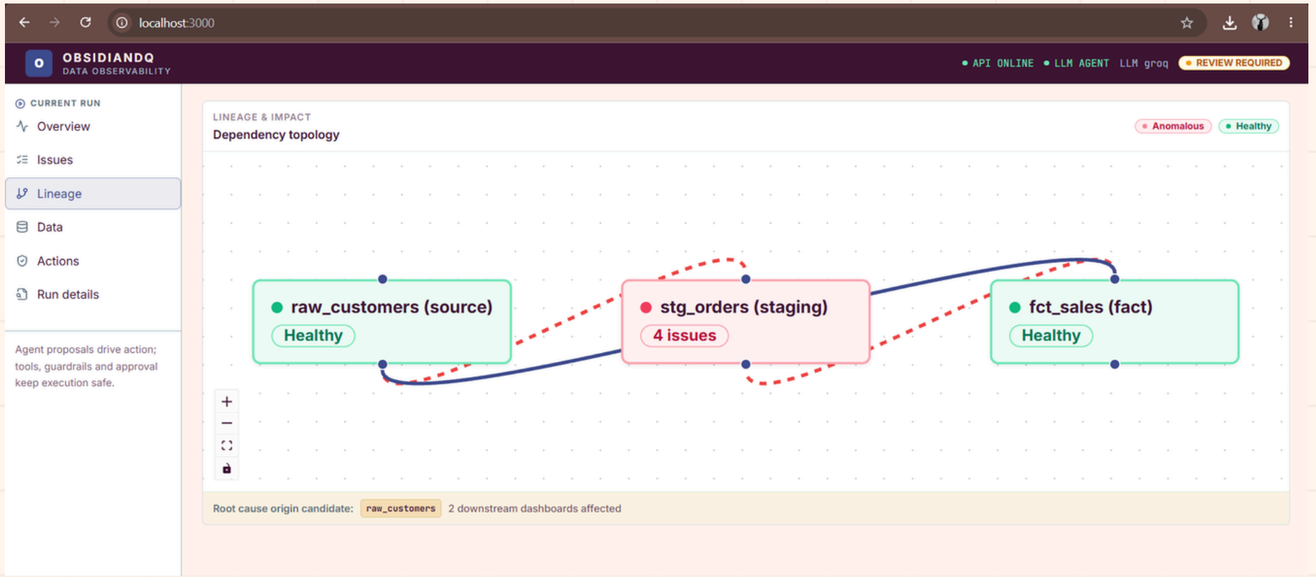
EXAMPLE PROCESS (WALKTHROUGH)

1. Detect - finds a rule violation: missing customer_id.
2. Trace - lineage graph traces it upstream to raw_customers.
3. Investigate - AI agent inspects bad rows and stats, read-only.
4. Diagnose - concludes "upstream problem" with a confidence score.
5. Plan - builds fixed sequence: Investigate → Heal → Quarantine → Verify.
6. Decide - low confidence → flagged for human review.
7. Check - Critic Agent reviews the plan for safety.
8. Pause - workflow halts, waits for human approval.
9. Approve - engineer approves; execution resumes.
10. Fix - SQL join repaired; bad rows quarantined, original file untouched.
11. Verify - re-checks cleaned data; reports success or honest failure.
12. Output - clean dataset + full audit trail, zero unapproved changes.

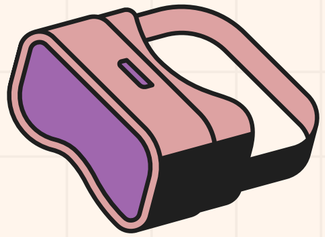
OUTPUT

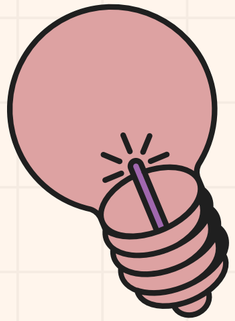

localhost:3000
ObsidianDQ
ENGINE V2.0
Deterministic & Agentic Data Quality Telemetry
System Online
FastAPI + LangGraph + DuckDB
Step 1 of 3 — Data Ingestion Source Configuration
ObsidianDQ Pipeline Ingestion
Upload your dataset, transformation query SQL, and lineage topology — or select our featured demo dataset.
Dataset File
Raw or staging data source
Upload File
.csv or .parquet
Transformation SQL
DuckDB transformation query
Upload File
.sql file
Lineage Topology
Upstream dependency graph
Upload File
.json topology
OR SELECT FEATURED DEMO
FEATURED DEMO
Synthetic E-Commerce Dataset
Includes pre-configured raw_customers.csv · stg_orders.parquet (with nulls & negative prices) · fct_sales.sql


localhost:3000
OBSIDIANDQ
DATA OBSERVABILITY
API ONLINE
LLM AGENT
LLM groq
REVIEW REQUIRED
CURRENT RUN
Overview
Issues
Lineage
Data
Actions
Run details
Agent proposals drive action; tools, guardrails and approval keep execution safe.
PIPELINE RUN
Data quality run
b15bdc0b…
REVIEW REQUIRED
Review issues →
Lineage impact
HEALTH SCORE
80 / 100
RECORDS SCANNED
100
raw_customers · stg_orders · fct_sales
DATA QUALITY ISSUES
4
BLAST RADIUS
2 downstream dashboards aff...
9/10 severity
SUMMARY
REVIEW REQUIRED
4 data-quality issues in stg_orders
The stg_orders stage failed due to four NOT_NULL violations—two missing customer_id and email values in stg_orders itself and two identical missing values propagated from the raw_customers source. This indicates a data ingestion or upstream transformation defect that fails to enforce mandatory fields; immediately add a pre-load validation step or modify the raw_customers extraction to flag and cleanse these nulls before they reach stg_orders.
ISSUES SUMMARY
What needs attention
4 total · select to inspect
ISSUE
RULE
SEVERITY
Missing customer id
1 rows · 1.0% · field customer_id
NOT_NULL
MEDIUM
Missing email
1 rows · 1.0% · field email
NOT_NULL
MEDIUM
Missing customer id
1 rows · 1.0% · field customer_id
NOT_NULL
MEDIUM
Missing email
1 rows · 1.0% · field email
NOT_NULL
MEDIUM
LINEAGE & IMPACT
Anomalous
Healthy
MULTI-AGENT RECOMMENDATION
3-agent graph


localhost:3000
OBSIDIANDQ
DATA OBSERVABILITY
API ONLINE
LLM AGENT
LLM groq
REVIEW REQUIRED
CURRENT RUN
Overview
Issues
Lineage
Data
Actions
Run details
Agent proposals drive action; tools, guardrails and approval keep execution safe.
LINEAGE & IMPACT
Dependency topology
Anomalous
Healthy
raw_customers (source)
Healthy
stg_orders (staging)
4 issues
fct_sales (fact)
Healthy
Root cause origin candidate:
raw_customers
2 downstream dashboards affected

TEAM
OBSIDIAN





THANK YOU